

Legally Backend — System Design Document

Version: 1.0

Scope: Spring Boot API (`legally-backend`)

Author: Bashir Mufol (portfolio)

1. Purpose

This document describes the architecture of the Legally backend: how HTTP requests flow, how legal and contact research run through a multi-provider AI chain, and how the API integrates with PostgreSQL, Firebase, Google Cloud, and external LLM services.

Legally provides **general legal information**, not legal advice. The backend accepts multimodal input, resolves jurisdiction, returns cited guidance where possible, and persists session-scoped consultation history.

2. System context

The React web client calls the Spring Boot API over HTTPS with a Firebase ID token and an optional session header (`X-Legally-Session-Id`). The API is stateless at the HTTP layer; user and session state live in PostgreSQL.

External dependencies:

Dependency	Role
------------	------

Cloud Run	Hosts the Spring Boot container
Cloud SQL (PostgreSQL)	Users, sessions, consultations, upload metadata
Firebase Auth	Anonymous sign-in; JWT verification on the API
Firebase Storage	Production file uploads
Gemini API	Legal analysis, contacts, media digest, Google Search grounding
Google Speech-to-Text	Voice transcription (separate from Gemini quota)
Groq, OpenRouter, Mistral, Cloudflare, Hugging Face	Optional fallback LLMs (OpenAI-compatible APIs)

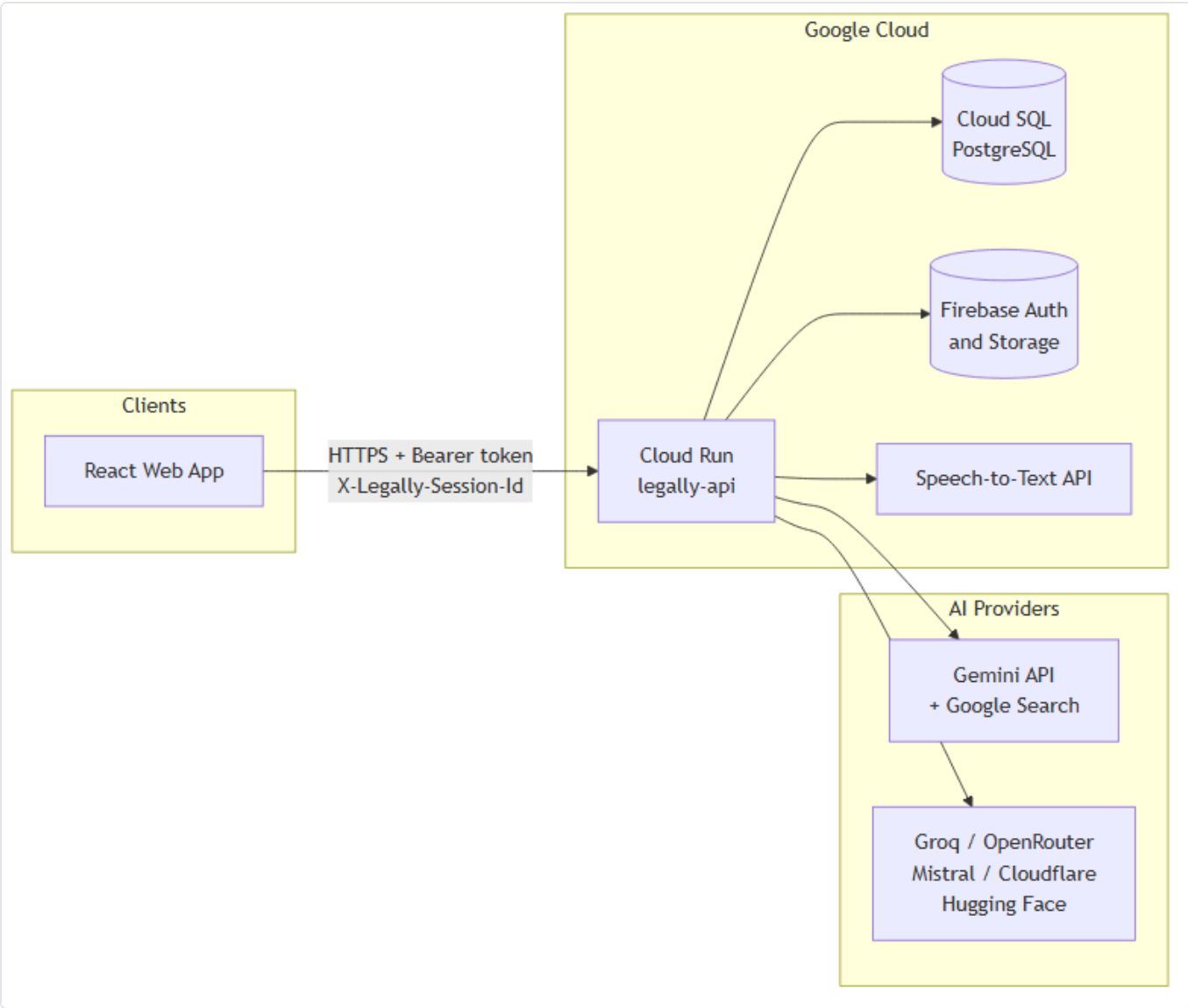


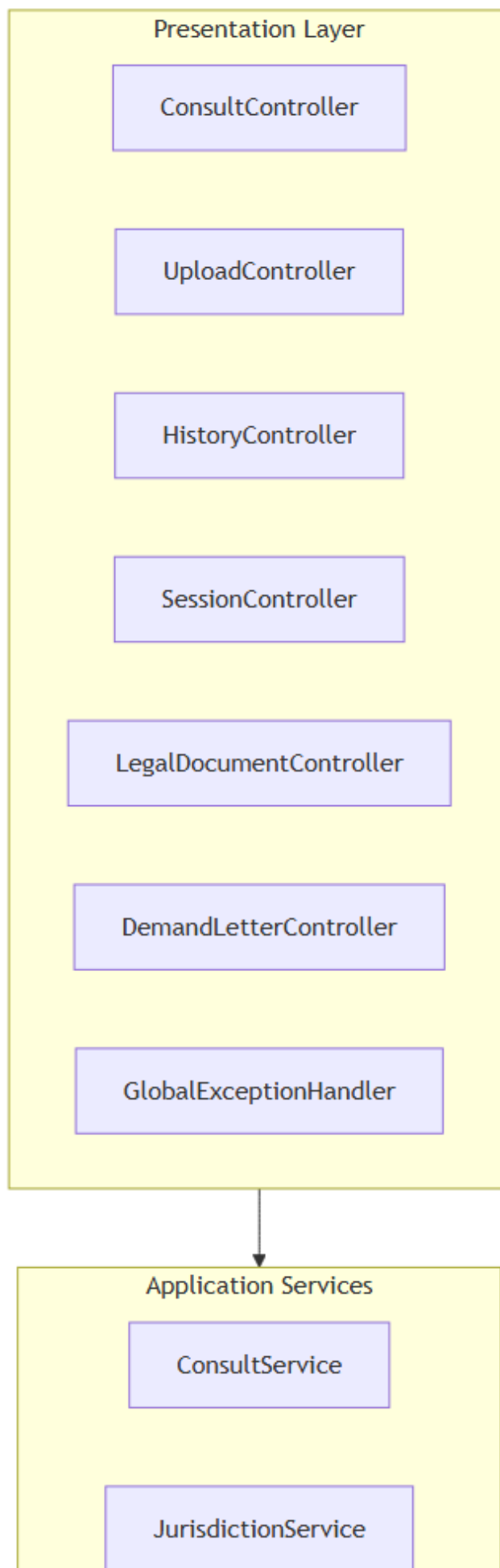
Figure 1: System context — clients, API, data stores, and external services.

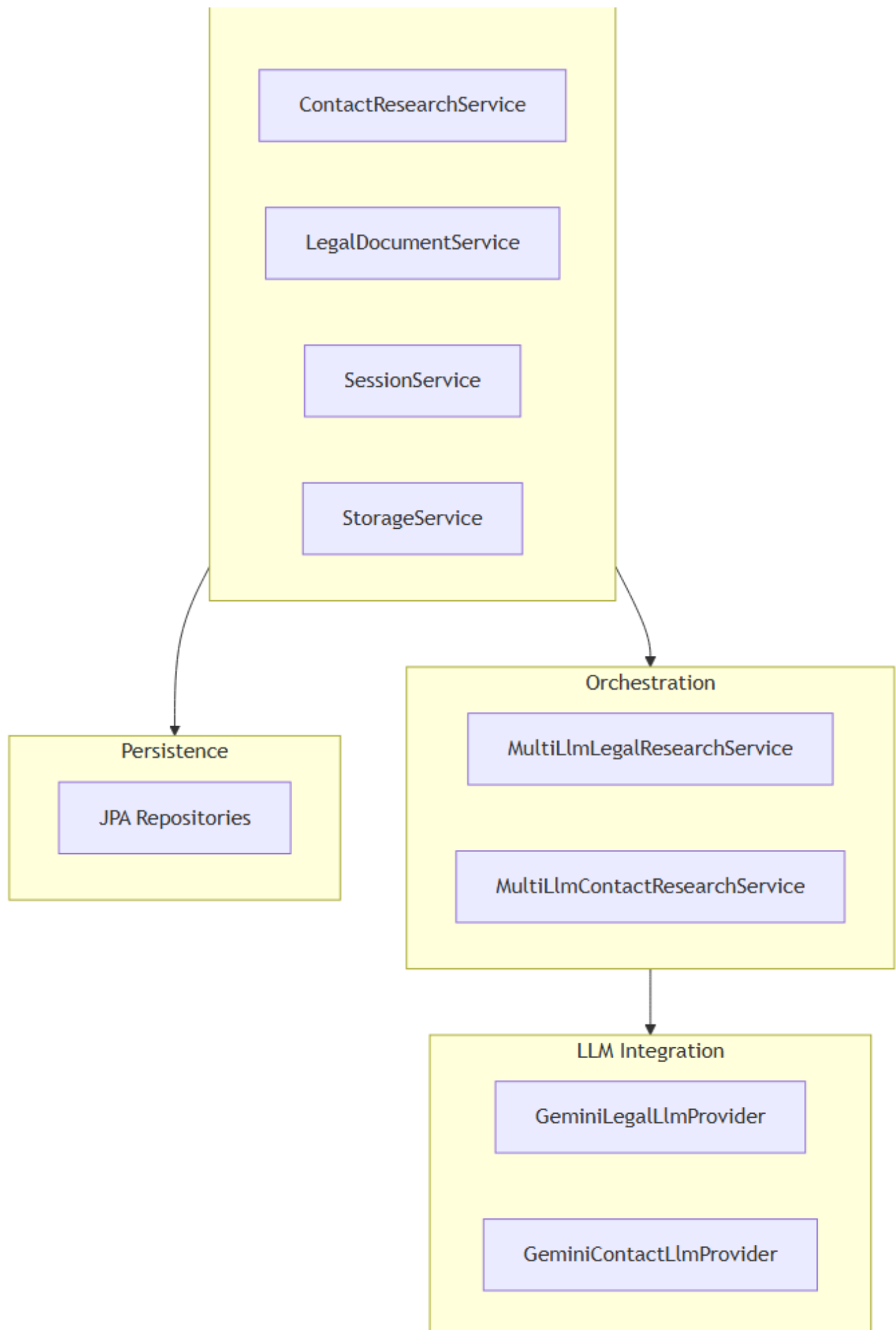
3. Goals and constraints

Goal	Implementation
Jurisdiction-aware answers	`JurisdictionService`, text parsing (Nigeria/US states, city phrases), optional Gemini detection
Resilience when one AI fails	Ordered `LegalLlmProvider` and `ContactLlmProvider` chains
Multimodal consults	Uploads, Gemini media digest, Speech-to-Text, Gemini native multimodal
No fabricated law on total failure	Explicit no-information and media-failed responses
Session privacy	Configurable TTL (default 72h), scheduled cleanup, new-session wipe
Secrets not in source	API keys via environment variables / `.env`

Non-goals: Billing, lawyer marketplace, static statute database. Legal text is generated at request time.

4. Layered architecture





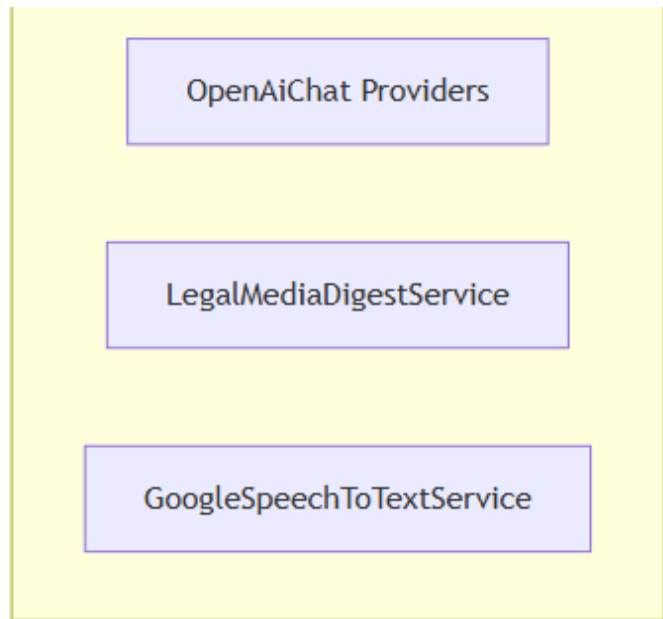


Figure 2: Layered backend architecture from REST controllers through persistence.

REST controllers

Controller	Base path	Purpose
`ConsultController`	`/api`	`POST /consult`
`UploadController`	`/api/uploads`	Multipart uploads
`HistoryController`	`/api/history`	Consultation history
`SessionController`	`/api/session`	New session / cleanup
`LegalDocumentController`	`/api/documents`	Draft agreements/letters
`DemandLetterController`	`/api/demand-letter`	Demand letters

5. Request pipeline (security)

Every `/api/**` request passes through filters before controllers:

1. **GeminiQuotaCircuitBreakerFilter** — Resets per-request ThreadLocal breakers for Gemini and OpenRouter; clears them in `finally`.
2. **FirebaseAuthenticationFilter** — Validates `Authorization: Bearer <Firebase JWT>` or assigns guest principal when Firebase is disabled.
3. **SessionHeaderFilter** — Binds `X-Legally-Session-Id` for scoping history and uploads.
4. **Spring Security** — Stateless; `CSRF` disabled for API usage.

Public routes: `/actuator/health`, `/actuator/info`, `OPTIONS`, and local `GET /api/uploads/files/**`.

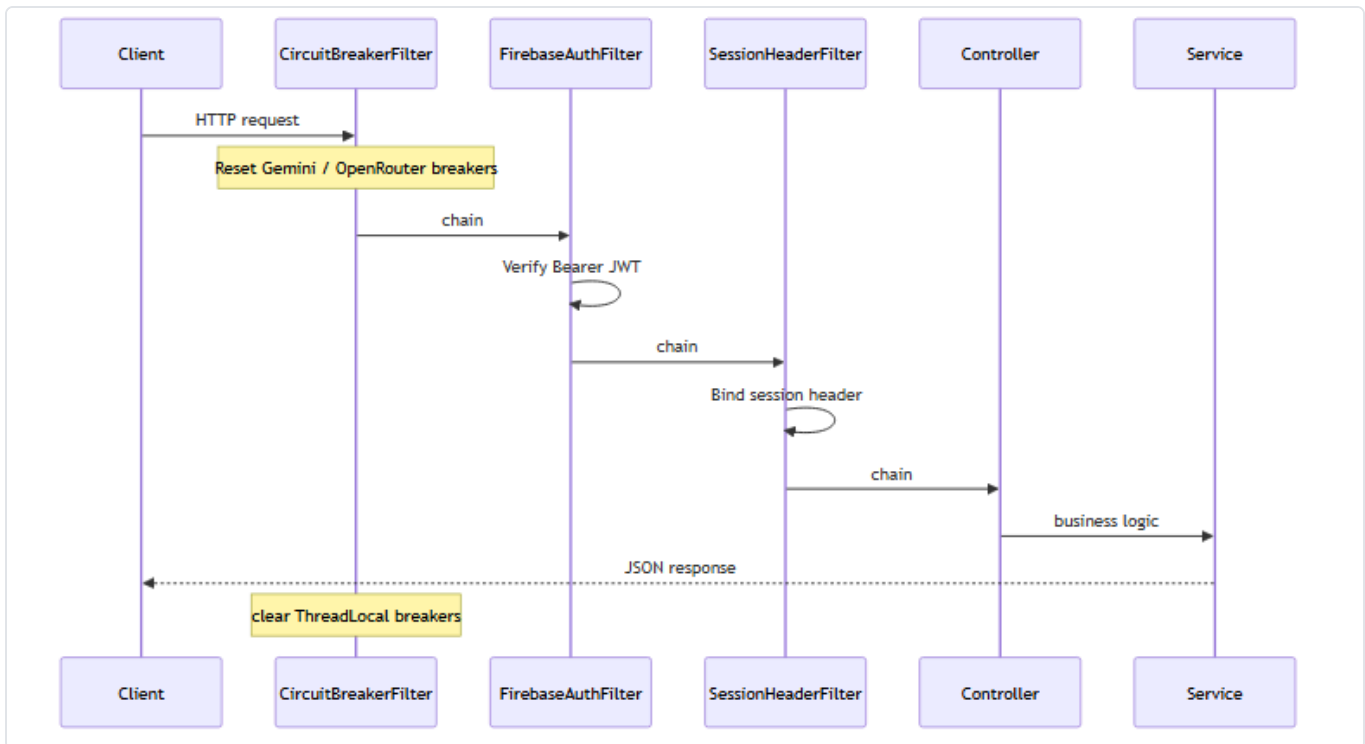


Figure 3: Security filter chain for `/api/**` requests.

6. Consult flow (core workflow)

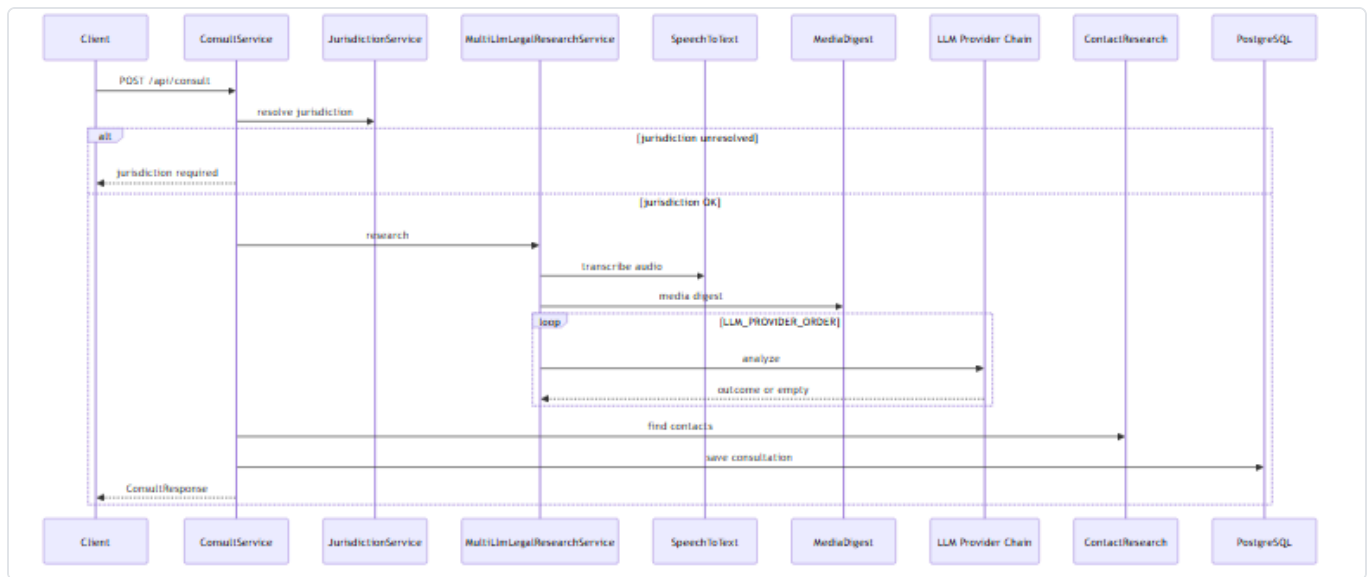


Figure 4: End-to-end consult workflow from `POST /api/consult` to response.

7. Multi-LLM legal provider chain

Providers implement `LegalLlmProvider`. Spring wires an ordered `List<LegalLlmProvider>` from `LegalLlmConfiguration` using `LLM_PROVIDER_ORDER` (default includes `gemini` first).

Typical order: `gemini` → `groq` → `openrouter` → `mistral` → `cloudflare` → `huggingface` (only configured providers are added).

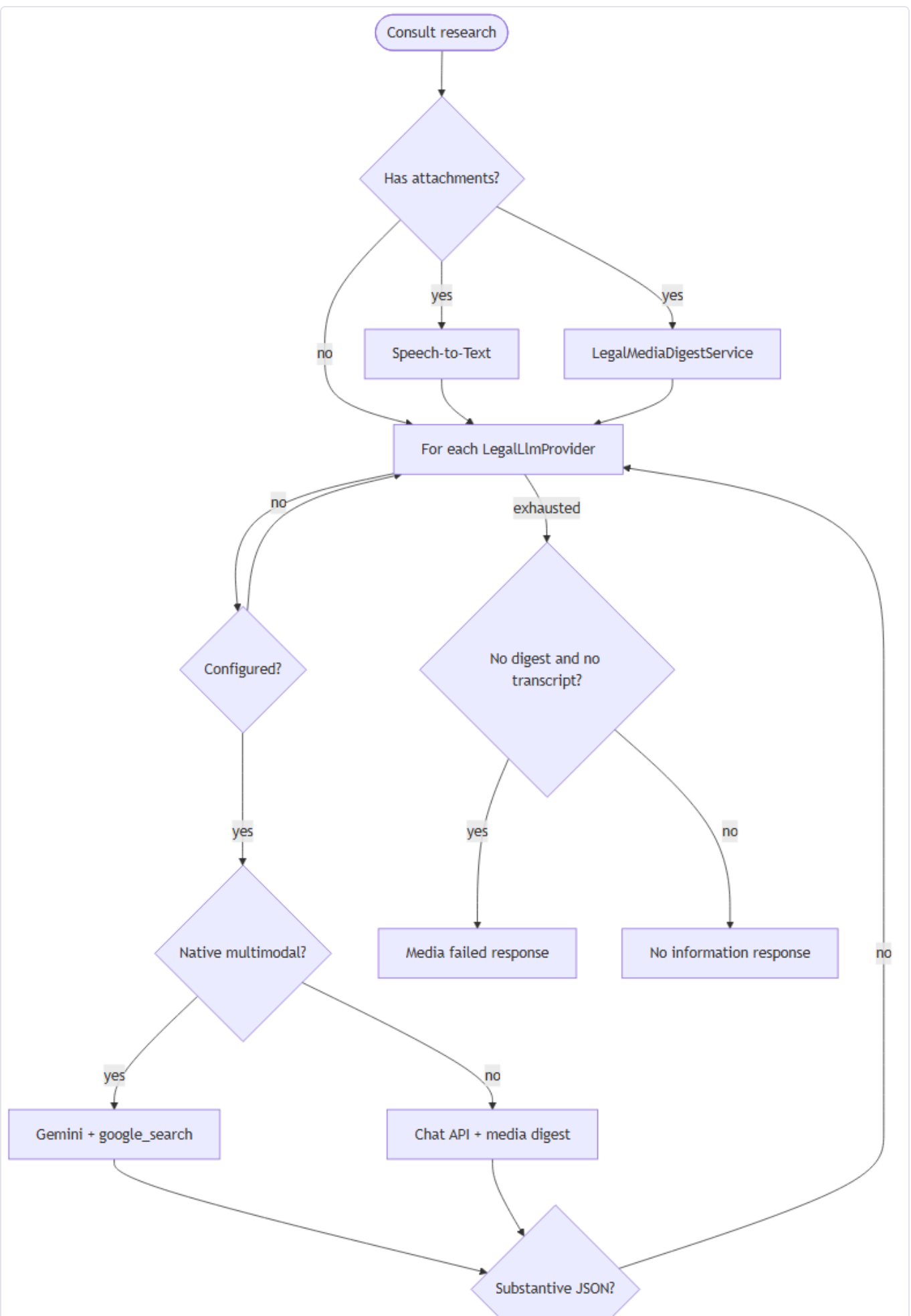
Per provider:

- Skip if not configured (missing API key/model).
- If `supportsNativeMultimodal()` is false: send text prompt with digest; do not attach raw bytes.
- If true (`GeminiLegalLlmProvider`): attach media parts; enable Google Search grounding.
- Accept result only if `LlmResponseParser.hasSubstantiveLegalContent()`.

After chain exhaustion:

- If attachments exist but both digest and transcript are empty → media processing failed response.
- Otherwise → no-information response with suggested next steps (may still fetch contacts).

Contact research uses the same pattern via `ContactLlmProvider` and `ContactResponseParser` validation rules.



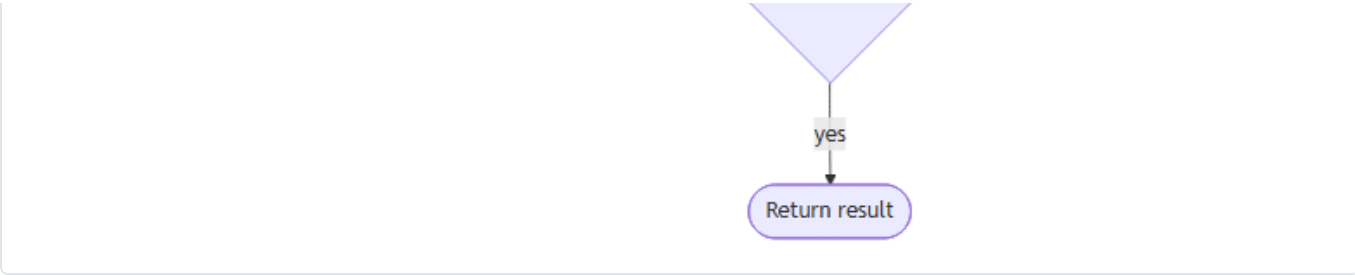


Figure 5: Ordered legal and contact LLM failover chains.

8. Jurisdiction resolution

1. Parse user message for country/state/city (`NigerianJurisdictionData` , `UsJurisdictionData` , `SubregionFromText` , country regexes).
2. Else use device location fields from request body.
3. Else optional `GeminiService.detectJurisdictionFromInputs` when user clearly names another place in text or media.
4. If still unresolved, return dedicated response (no legal LLM run).

Resolved jurisdiction drives prompts, API response metadata (`jurisdictionCountry` , `jurisdictionRegion` , `locationSource`), and contact research.

9. Data model (PostgreSQL)

Entity	Purpose
`AppUser`	Firebase UID
`UserSession`	Session ID, last activity
`ConsultationRecord`	Scenario + full response JSON
`MediaUploadRecord`	Storage path, MIME type, session

`DemandLetterRecord`	Generated demand letter content
----------------------	---------------------------------

Relationships: User → many Sessions → many Consultations / Uploads / Demand letters.

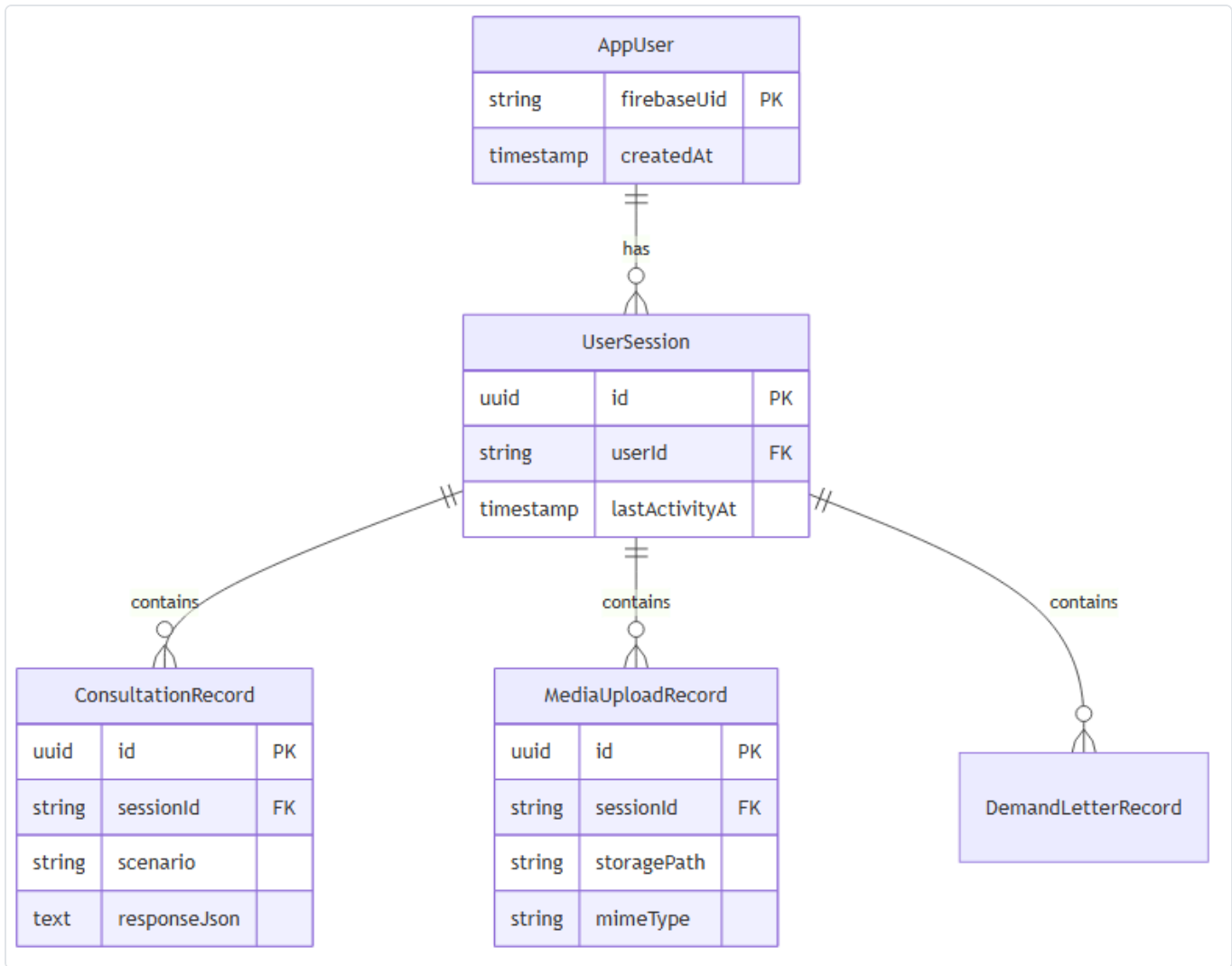


Figure 6: Core PostgreSQL entities and relationships.

10. File storage

`StorageService` stores uploads in **Firestore** (`firebase://` paths) when Firestore is enabled, else **local disk** under `backend/uploads/`. LLM services read bytes via `StorageService.readBytes()` for digest and Gemini multimodal calls.

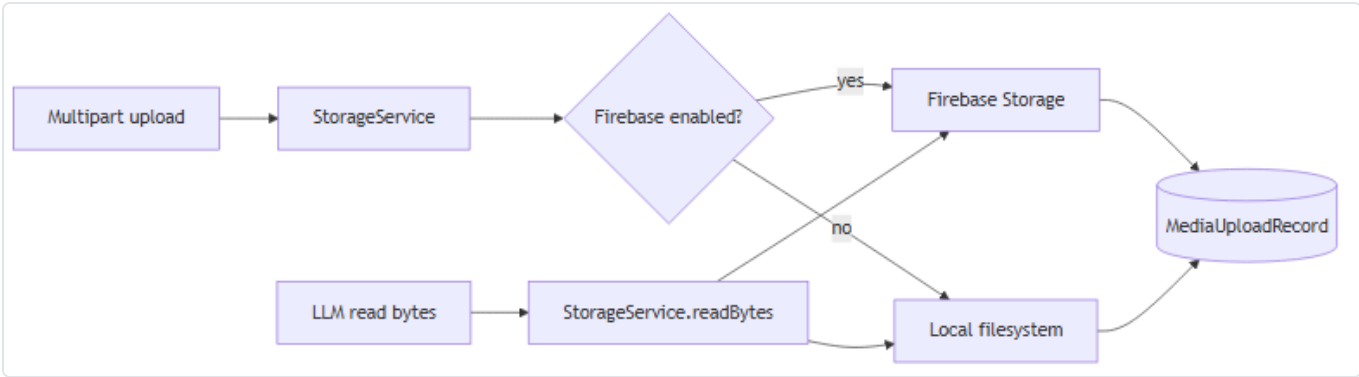


Figure 7: Upload storage paths and read flow for LLM services.

11. Deployment (production)

Component	Technology
API	Cloud Run (Docker image from <code>`backend/Dockerfile`</code>)
Database	Cloud SQL PostgreSQL via socket factory (<code>`legally.database.mode=cloud-sql`</code>)
Auth / files	Firebase
Frontend	Static hosting (separate); <code>`CORS_ALLOWED_ORIGINS`</code> on API

Configuration is externalised through `LegallyProperties` and environment variables (`GEMINI_API_KEY`, `LLM_PROVIDER_ORDER`, `DATABASE_URL`, etc.).

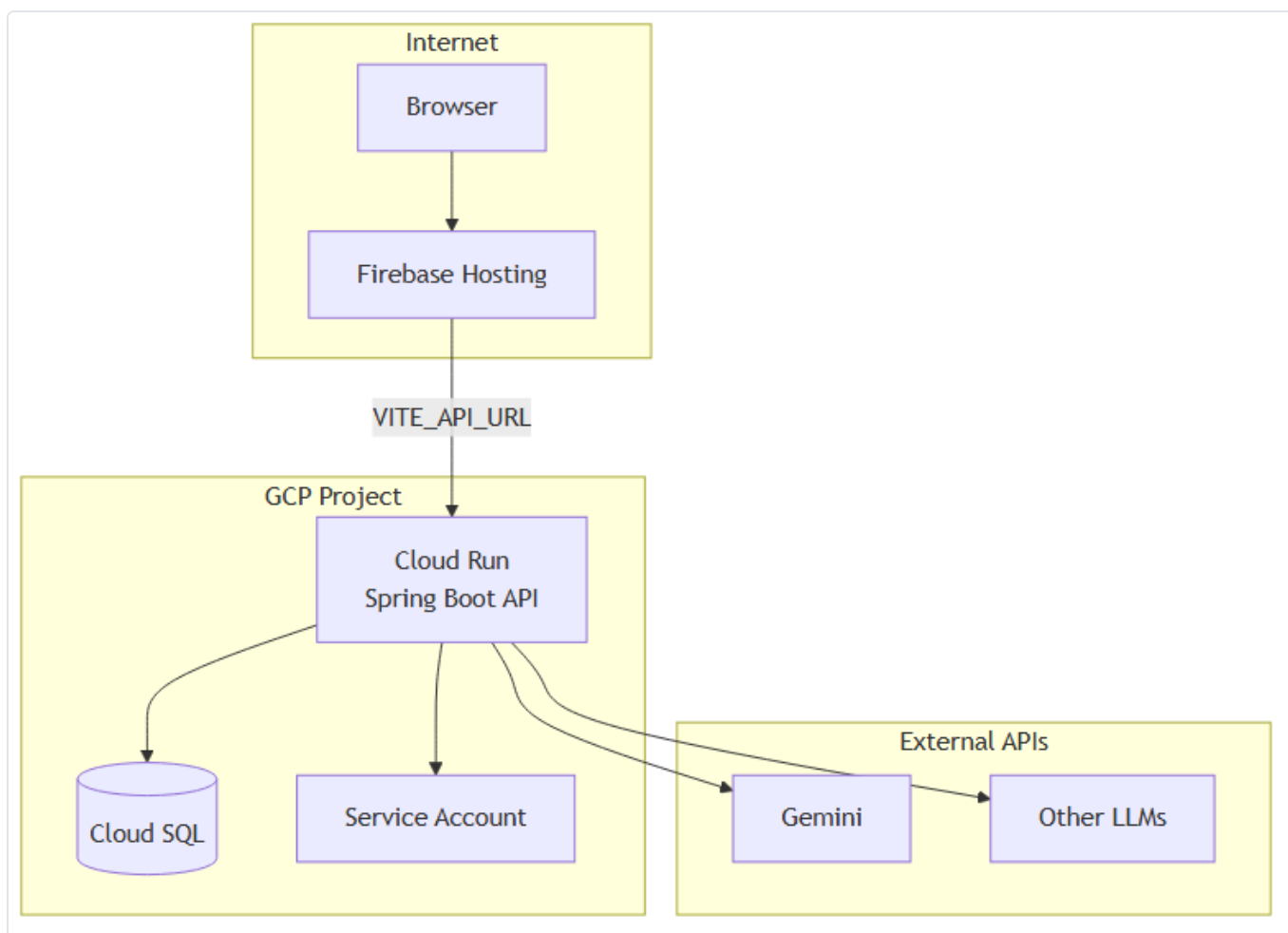


Figure 8: Production deployment topology on Google Cloud.

12. Failure handling

Scenario	Behaviour
Gemini 429	Per-request circuit breaker; skip further Gemini in same request
OpenRouter 429	`OpenRouterRateLimitCircuitBreaker`; no HTTP auto-retry
All legal providers fail	No-information response; contacts may still run
Media unreadable	Media processing failed response
Unknown jurisdiction	Jurisdiction unresolved response

13. Technology stack

- Java 21, Spring Boot 4
 - Spring Web, Security, Data JPA, Validation, Actuator
 - PostgreSQL, HikariCP, Cloud SQL socket factory
 - Firebase Admin SDK
 - RestClient (Apache HttpClient 5, automatic retries disabled on 429)
 - Jackson, Jsoup
 - JUnit 5
 - Maven, Docker
-

14. Design highlights for reviewers

1. **Provider pattern** with ordered failover for legal and contact chains.
 2. **Clear separation** between application services and `llm` integration package.
 3. **Per-request circuit breakers** to avoid hammering rate-limited APIs within one consult.
 4. **Dual multimodal path:** Gemini native vs digest + text-only fallbacks.
 5. **First-class degraded responses** for jurisdiction, media, and no-information cases.
 6. **Session-scoped retention** with scheduled purge.
-

Document generated for portfolio use. Reflects the `com.legally` backend codebase.